Assignment 1 Work Log

MapReduce-Based Taxi Fleet and Company Performance Analysis

Field	Details
Student	Samuel Klefe
Student ID	s4271512
Course	COSC2637 Big Data Processing
Program	Postgraduate
Submission	Individual
Prepared	28 August 2026

Purpose: I use this work log to explain how data moves through each mapper, shuffle/sort stage, reducer, iteration, and chained job. The submitted source code is provided separately in the code ZIP and Turnitin PDF.

Section 1 - Individual Contribution

I am submitting this assignment individually, so the group member contribution record is not applicable. Section 2 explains my design for all three postgraduate tasks, and Section 3 records the required submission declaration checks.

Section 2 - Step-by-Step MapReduce Job Design

I used Python programs with Hadoop Streaming only. For every stage below, I identify the input record, the mapper's emitted <key, value> pair, the partition/shuffle/sort behaviour, the reducer input and output, and any HDFS handoff to the next job. The notation <key, value> shows the logical Streaming boundary; the submitted programs encode these fields with tab characters. Every MapReduce job is configured with exactly three reducers.

Task 1 - Taxi-Level Trip Efficiency Analysis

Step 1 - Read and classify mapper input

I classify each trip as long when distance is at least 200, medium when distance is at least 100 but less than 200, and short when distance is less than 100. I treat the pair (taxi ID, trip type) as the primary aggregation key.

```
Mapper input: trip_id,taxi_id,fare,distance,pickup_x,pickup_y,dropoff_x,dropoff_y
```

For each valid row, I extract the taxi ID, fare, and distance. I convert the fare to integer cents so that mapper split boundaries and binary floating-point addition cannot alter the final two-decimal result.

Step 2 - In-mapper combine and emit <key, value>

I preserve one dictionary for the mapper's complete input split. Each entry stores the partial count, maximum cents, minimum cents, and sum of cents for one (taxi ID, trip type) key. I therefore emit one partial pair per distinct key held by that mapper instead of one pair per trip.

```
Mapper emits: <taxi_id<TAB>trip_type,
partial_count<TAB>partial_max_cents<TAB>partial_min_cents<TAB>partial_sum_cents>

Example: 5,64,378.31,202.95,... -> <64<TAB>long, 1<TAB>37831<TAB>37831<TAB>37831>
```

Step 3 - Partition, shuffle, and sort

I set stream.num.map.output.key.fields=2 and use KeyFieldBasedPartitioner, so the complete (taxi ID, trip type) pair is the partition key. Hadoop sends all partial values for that pair to one reducer and sorts identical composite keys consecutively. No custom comparator is needed because the two-field key already defines the required grouping and order.

Step 4 - Reduce each key group and write final output

```
Reducer consumes: <taxi_id<TAB>trip_type, [partial_count, partial_max_cents,
partial_min_cents, partial_sum_cents]...>
```

The reducer streams one key group at a time, adds partial counts and sums, takes the maximum of partial maxima and minimum of partial minima, and divides total cents by 100 times the trip count using explicit half-up rounding.

```
Reducer writes:
taxi_id<TAB>trip_type<TAB>total_trips<TAB>max_fare<TAB>min_fare<TAB>average_fare
```

Execution contract: The job reads /Input/Trips.txt, writes /Output/task1, runs with exactly three reducers, and emits six tab-separated fields with every fare value formatted to two decimal places.

Task 2 - PAM Clustering by Drop-off Location

Step 1 - Read the iteration limit and initial medoids

I implement the specified PAM process. The first non-empty line of initialization.txt gives the iteration limit v; each remaining line gives one initial medoid, so their count determines k. My driver reads only this control file and the bounded cluster summaries, never the complete trip dataset.

```
Iteration flow: /Input/Trips.txt -> mapper assignment -> shuffle by cluster -> reducer
PAM update -> /tmp/bdp_assignment1_task2/iteration_NNN -> driver -> next medoid file
```

Step 2 - Map trips to their nearest current medoid

```
Mapper input: trip_id,taxi_id,fare,distance,pickup_x,pickup_y,dropoff_x,dropoff_y
```

At each iteration I distribute task2_medoids.txt. The mapper extracts (dropoff_x, dropoff_y) and chooses the lowest-index medoid with minimum squared Euclidean distance; the square root is unnecessary because it does not change the ordering.

Step 3 - Combine repeated points and emit <key, value>

I combine repeated coordinates per cluster and preserve multiplicity with a count. I flush after 50,000 distinct keys, so the mapper never assumes every coordinate fits in memory.

```
Mapper emits: <cluster_index, dropoff_x<TAB>dropoff_y<TAB>partial_count>

Example with the supplied medoids: drop-off (5.020, 31.765) -> <1,
5.020<TAB>31.765<TAB>1>
```

Step 4 - Shuffle by cluster and perform the PAM update

The cluster index is the Streaming key, so the default hash partitioner sends one cluster to one reducer and the default sort keeps its records consecutive. No custom comparator is required.

```
Reducer consumes: <cluster_index, [(dropoff_x, dropoff_y, partial_count), ...]>
```

The reducer merges counts into weighted distinct points and evaluates every assigned point as replacement candidate o. It divides o's weighted sum of Euclidean distances by the assigned-trip count; the candidate with the lowest average becomes the new medoid.

```
Reducer emits: <cluster_index,
new_x<TAB>new_y<TAB>#points<TAB>avg_dissimilarity<TAB>membership_signature>
```

A deterministic SHA-256 digest of sorted (x, y, count) entries lets the driver detect unchanged assignments without retrieving every trip assignment.

Step 5 - Control iteration, convergence, and final output

1. I remove stale final and temporary HDFS paths.
2. I run one three-reducer job at /tmp/bdp_assignment1_task2/iteration_NNN.
3. I read one summary per cluster, print the required iteration fields, update medoids, and compare assignment signatures.
4. I stop at convergence or v, publish exactly k records to /Output/task2, and remove temporary HDFS output.

```
Final output: medoid_x<TAB>medoid_y<TAB>#points<TAB>avg_dissimilarity
```

Task 3 - Company Performance Analysis

I use three separate MapReduce jobs: a reduce-side join, company aggregation, and global revenue sorting. Each job launches exactly three reducers. The submitted scripts read both source datasets directly from HDFS and keep intermediate results under /tmp/bdp_assignment1_task3 until the downstream job succeeds.

```
/Input/Taxis.txt + /Input/Trips.txt -> Job 1 join -> Job 2 aggregation -> Job 3 global
sort -> /Output/task3
```

Step 1 - Job 1 mapper tags both join inputs

The mapper distinguishes the datasets by field count. I tag taxi metadata with 0 and trip details with 1. The first two fields form a composite key.

```
Taxis mapper emits: <taxi_id<TAB>0, company_id>

Trips mapper emits: <taxi_id<TAB>1, fare<TAB>distance>
```

Step 2 - Job 1 partitions, secondary-sorts, and joins

I define the first two fields as the composite mapper key, partition with -k1,1 on taxi ID, and compare with -k1,1n -k2,2n. This sends all records for one taxi to one reducer and sorts tag 0 metadata before tag 1 trips. The reducer consumes <taxi_id<TAB>tag, value> groups, stores only the current company ID, emits one fleet marker, and streams joined trip records without buffering a trip list.

```
Job 1 reducer writes: company_id<TAB>taxi_id<TAB>F<TAB>0.00<TAB>0.00

Job 1 reducer writes: company_id<TAB>taxi_id<TAB>T<TAB>fare<TAB>distance
```

The fleet marker ensures a taxi with no trips still contributes to fleet size. Job 1 output is stored at /tmp/bdp_assignment1_task3/job1_join and becomes Job 2 mapper input.

Step 3 - Job 2 mapper partially aggregates Job 1 output

The mapper maintains bounded partial state for each (company ID, taxi ID) pair and periodically flushes after 50,000 keys. It emits fleet presence, partial revenue, partial trip count, and partial distance.

```
Job 2 mapper emits: <company_id<TAB>taxi_id,
fleet_present<TAB>partial_revenue<TAB>partial_trips<TAB>partial_distance>
```

Step 4 - Job 2 secondary-sorts taxis and reduces companies

I partition with -k1,1 on company ID and compare with -k1,1n -k2,2n on company then taxi. The reducer consumes <company_id<TAB>taxi_id, partial metrics> in taxi order, merges all partials for one taxi, and counts fleet members by ordered key changes rather than storing an unbounded set. For each company, I calculate revenue per taxi as total revenue divided by fleet size and average distance as total distance divided by total trips.

```
Job 2 reducer writes:
company_id<TAB>total_revenue<TAB>total_trips<TAB>fleet_size<TAB>revenue_per_taxi<TAB>aver
age_distance
```

Job 2 output is stored at /tmp/bdp_assignment1_task3/job2_aggregation and becomes Job 3 mapper input.

Step 5 - Job 3 globally sorts the bounded company table

A reverse comparator alone would sort only inside each reducer partition. To produce one globally descending result with three reducers without TotalOrderPartitioner, I replicate the bounded company summary table to bucket keys 0, 1, and 2. I partition by bucket and sort each replicated table by revenue descending with company ID as the numeric tie-breaker.

```
Job 3 mapper emits: <bucket<TAB>total_revenue<TAB>company_id,
trips<TAB>fleet<TAB>revenue_per_taxi<TAB>average_distance>
```

I partition with -k1,1 on bucket and compare with -k1,1n -k2,2nr -k3,3n, which sorts bucket ascending, revenue numerically descending, and company ID numerically ascending as a tie-breaker. Each reducer receives one copy of the same sorted company table. Bucket 0 emits the first rank third, bucket 1 the middle third, and bucket 2 the final third. For N companies, bucket b emits indices floor(bN/3) through floor((b+1)N/3)-1. Concatenating part-00000, part-00001, and part-00002 therefore produces the required global descending sequence. Replication occurs only after aggregation, so the large trip dataset is not replicated.

Step 6 - Write the assessed Task 3 output

```
Job 3 reducer writes:
company_id<TAB>total_revenue<TAB>total_trips<TAB>fleet_size<TAB>revenue_per_taxi<TAB>aver
age_distance
```

Job 3 writes this six-field format to /Output/task3 with monetary and average values at two decimal places. The reducer output is already globally ordered by descending total revenue, so task3-run.sh performs no Bash post-processing sort. The script removes the two intermediate HDFS paths only after the downstream jobs succeed.

Section 3 - Submission Declaration Checklist

Submission requirement	Status
Individual submission	Complete
Tasks 1, 2, and 3 included for COSC2637	Complete
Python and Hadoop Streaming only; no MapReduce library	Complete
Exactly three reducers configured for every MapReduce job	Complete
Required case-sensitive HDFS input and output paths used	Complete
All task run scripts execute end to end and clear stale outputs	Complete
Mapper/reducer interfaces and all intermediate formats explained	Complete
Code and shell scripts include concise technical comments	Complete
Flat ZIP contains all required source/configuration files	Complete
Standalone code PDF prepared for Turnitin	Complete
Work log prepared as a separate Word document	Complete
Submission files use the s4271512_PG filename base	Complete